Colorizer Samples

Tuesday, October 27, 2026 8:42 AM

Samples

The following pages show examples of every programming language supported by OneMore's *Colorize* command. These are not intended to be working or even sensible code examples. They are merely a way to exercise the syntax recognition patterns that define each language.

Arduino
AutoHotKey
Bash
Batch
C#
C++
CSS
Dockerfile
Go
Java
JavaScript
JSON
HTML
Onetastic
Pascal
PlantUML
PowerShell
PHP
Python
Registry
R
Regex
Rust
Scala
SQL
TypeScript
Verilog
Visual Basic
VHDL
Wolfram
XML
YAML

Arduino

Monday, October 26, 2026 5:13 PM

```
// plain line comment
/// doc comment on a line
/* block comment on one line */
/*
 * multi-line block comment
 */

#include <Wire.h>
#include <Servo.h>
#include "config.h"
#define LED_PIN 13
#define SENSOR_PIN A0
#define SQUARE(x) ((x) * (x))
#ifdef ARDUINO_AVR_UNO
#pragma message "building for Uno"
#endif

// Arduino constants and types
const byte BUTTON = 2;
const word TIMEOUT = 1000;
const int THRESHOLD = 0x1FF;
boolean ledOn = false;
String message = "hello";

namespace hw {
  struct Pin { byte number; byte mode; };
  enum class State { Idle, Busy };
}

class Blinker final {
public:
  explicit Blinker(uint8_t pin, unsigned long interval = 500UL);
  [[nodiscard]] bool update() const;
private:
  uint8_t _pin;
  unsigned long _last = 0, _interval;
  volatile bool _flag;
};

Servo servo;
int samples[8];
float readings[4] = {0.5f, 1.25f, 3.14159f, 2.0e-3f};
char label[] = "plain \"quoted\" string\n";
char grade = 'A';
const char* raw = R"(raw string with "quotes")";

void onButton();

// setup() and loop() are the standard Arduino entry points
void setup() {
  Serial.begin(9600);
  Wire.begin();

  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON, INPUT_PULLUP);
  pinMode(SENSOR_PIN, INPUT);
  attachInterrupt(digitalPinToInterrupt(BUTTON), onButton, CHANGE);

  servo.attach(9);
  randomSeed(analogRead(A1));
  analogReference(DEFAULT);
```

```
  Serial.println(F("ready"));
  Serial.println(message + " " + String(PI, 4));
}

void loop() {
  // numbers
  int a = 42;
  long b = 100000L;
  unsigned int c = 10u;
  float d = 3.14;
  double e = 6.02e23;
  int f = 0xFF;
  int g = 0b10100101;
  uint8_t h = 0x0F;
  int16_t i = -32768;
  size_t n = sizeof(samples) / sizeof(samples[0]);

  // digital and analog I/O
  digitalWrite(LED_BUILTIN, HIGH);
  delay(500);
  digitalWrite(LED_BUILTIN, LOW);
  delayMicroseconds(250);

  int raw1 = analogRead(SENSOR_PIN);
  int pwm = map(raw1, 0, 1023, 0, 255);
  pwm = constrain(pwm, 0, 255);
  analogWrite(LED_PIN, pwm);

  int pressed = digitalRead(BUTTON);
  unsigned long now = millis();
  unsigned long later = micros();
  long r = random(0, 100);

  tone(8, 440, 200);
  noTone(8);
  shiftOut(11, 12, MSBFIRST, 0xAA);
  unsigned long width = pulseIn(7, HIGH);

  // math constants
  float rad = 90 * DEG_TO_RAD;
  float deg = HALF_PI * RAD_TO_DEG;
  float tau = TWO_PI;

  // control flow
  if (pressed == LOW && !ledOn) {
    ledOn = true;
  } else if (now > later || TIMEOUT == 0) {
    ledOn = false;
  } else {
    return;
  }

  for (int k = 0; k < 8; k++) {
    samples[k] = analogRead(A0);
    if (samples[k] > THRESHOLD) { continue; }
    if (k > 5) { break; }
  }

  while (Serial.available() > 0) {
    char ch = Serial.read();
    switch (ch) {
      case 'a': servo.write(0); break;
      case 'b': servo.write(90); break;
      default: break;
    }
  }

  do { a++; } while (a < 5);

  bool *nothing = nullptr;
```

```
  auto flag = true;
  (void)nothing; (void)flag;
}

void onButton() {
  detachInterrupt(digitalPinToInterrupt(BUTTON));
}

bool Blinker::update() const {
  return millis() - _last >= _interval;
}
```

AutoHotKey

Monday, November 28, 2022 10:15 PM

```
; plain line comment
/* block comment on one line */
/*
   multi-line block comment
*/

#NoEnv
#SingleInstance Force
#Persistent
#Include lib.ahk

; hotkeys
^!n::Run, notepad.exe
#c::Send, ^c
~LButton & RButton::MsgBox, chord
F1::
    MsgBox, Help
return

; hotstrings
::btw::by the way
:*:addr::123 Main Street

; numbers
count := 42
hex := 0xFF
pi := 3.14159

; strings and variables
name := "plain ""quoted"" string"
msg = unquoted legacy text
greeting := "Hello, " . name . "!"
MsgBox, % "Value: " count
MsgBox, Hello %name%, you are %A_UserName% on %A_ComputerName%
path := A_ScriptDir "\data" A_Space A_Tab A_Now

; keys
Send, {Enter}{Tab}{Esc}{Left 3}
Send, ^{Home}
Send, {Ctrl down}c{Ctrl up}

; control flow
if (count > 0 && true)
{
    MsgBox, positive
}
else if (count < 0 || false)
{
    MsgBox, negative
}
else
{
    return
}

Loop, 10
{
    if (A_Index = 5)
        continue
    if (A_Index > 8)
        break
}
```

```
while (count > 0)
    count--

for key, value in myArray
    MsgBox, %key%: %value%

try
{
    FileRead, content, missing.txt
}
catch e
{
    MsgBox, % e.message
}
finally
{
    count := 0
}

switch count
{
    case 1: MsgBox, one
    default: MsgBox, other
}

Gosub, Cleanup
Goto, Done

Cleanup:
    SetTimer, Tick, 1000
return

Done:
    ExitApp

MyFunc(a, b := 2)
{
    global count
    static calls := 0
    local temp := a + b
    return temp
}

class Widget
{
    __New(name) {
        this.name := name
    }
}
```

Bash

Monday, October 26, 2026 5:28 PM

```
#!/usr/bin/env bash
# plain line comment

set -euo pipefail

NAME="world"
COUNT=10
readonly CONFIG='single quoted $notExpanded'
local_var="with \"escaped\" quotes"

# known and user variables
echo "$HOME $PWD $USER $SHELL $PATH $0 $$ $# $@ $* $?"
echo "${NAME} and ${COUNT:-5} and $NAME and $undefined_thing"

# numbers
let total=42+7
port=8080

greet() {
    local who="$1"
    echo "Hello, $who"
    return 0
}

function cleanup {
    unset NAME
    exit 1
}
trap cleanup EXIT

# control flow
if [ "$COUNT" -gt 5 ] && [ -n "$NAME" ]; then
    printf '%s\n' "big"
elif [[ $COUNT -eq 0 ]]; then
    echo "zero"
else
    echo "small"
fi

for i in 1 2 3; do
    if [ $((i % 2)) -eq 0 ]; then continue; fi
    if [ "$i" -gt 2 ]; then break; fi
    echo "$i"
done

while read -r line; do
    echo "$line"
done < input.txt

until [ "$COUNT" -le 0 ]; do
    COUNT=$((COUNT - 1))
done

case "$NAME" in
    world) echo "hi" ;;
    *) echo "unknown" ;;
esac

select opt in a b c; do break; done
time sleep 1
cd /tmp && pwd
export PATH="$PATH:/opt/bin"
```

```
source ./lib.sh
alias ll='ls -l'
eval "$(echo hi)"
exec bash
```

Batch

Monday, October 26, 2026 5:43 PM

```
@echo off
REM plain remark
:: double-colon comment
rem another remark
setlocal enabledelayedexpansion

set NAME=world
set /a COUNT=42+7
set "QUOTED=has spaces"
echo Hello %NAME% from %~dp0 and %1 %* !COUNT!
echo "quoted string" & echo done

:start
if "%1"=="" goto :usage
if not exist "%~f1" (
    echo File missing
    exit /b 1
) else (
    type "%~f1" | find /i "error" > out.txt 2>&1
)

for %%f in (*.txt) do echo %%f
for /l %%i in (1,1,10) do if %%i equ 5 goto :end
if errorlevel 1 call :helper arg1
if defined NAME (echo defined) else (echo undefined)
if %COUNT% gtr 10 echo big
if %COUNT% lss 5 echo small

:helper
    copy /y src dest
    xcopy /s /e src dest
    robocopy a b /mir
    cd /d "%TEMP%"
    pushd .. & popd
    exit /b 0

:usage
    echo Usage: %~nx0 file
:end
endlocal
pause
```

C#

Monday, October 26, 2026 4:42 PM

```
// plain line comment
/// <summary>XML doc comment with <see cref="Widget"/> tag</summary>
/* block comment on one line */
/*
 * multi-line block comment
 */

#nullable enable
#region Usings
#define TRACE
#if DEBUG
#warning debug build
#endif
#pragma warning disable CS0168
#endregion

using System;
using System.Collections.Generic;

[assembly: AssemblyTitle("Sample")]
[module: SuppressMessage("Style", "IDE0001")]

namespace Demo.Core
{
    [Serializable]
    [Obsolete("old", false)]
    public abstract class Widget<T> : Base, IDisposable where T : class, new()
    {
        public enum Color { Red, Green, Blue }
        public struct Point { public int X; public int Y; }
        public interface IThing { void Run(); }
        public record Person(string Name, int Age);
        public record struct Pair(int A, int B);
        public delegate void Handler(object sender, EventArgs e);

        private const int Max = 0x7FFF_FFFF;
        private static readonly string Name = "plain \"quoted\" string\n";
        protected internal volatile bool flag;

+       public event EventHandler? Changed;

        [STAThread]
        public static async Task<int> Main(string[] args)
        {
            // numbers
            int a = 42;
            long b = 100_000L;
            uint c = 10u;
            ulong d = 10UL;
            float e = 1.5f;
            double f = 6.02e23;
            decimal g = 19.99m;
            int h = 0xDEADBEEF;
            int i = 0b1010_0101;

            // chars and strings
            char ch = 'x';
            char esc = '\n';
            char uni = 'A';
            string s1 = "plain";
            string s2 = @"verbatim ""quoted"" string
                spanning lines";
            string s3 = $"interpolated {a} and {b:N2}";
            string s4 = $@"interpolated verbatim {c}";
```

```
            string s5 = """
                raw string literal
                with "quotes" inside
                """;

            // literals and types
            object o = null;
            bool t = true, u = false;
            dynamic dyn = new { X = 1 };
            var list = new List<string>();
            nint ptr = 0;
            sbyte sb = 1; byte by = 2; short sh = 3; ushort us = 4;

            // control flow
            if (a > 0 && t)
            {
                Console.WriteLine("positive");
            }
            else if (a < 0 || u)
            {
                throw new InvalidOperationException(nameof(a));
            }

            for (int n = 0; n < Max; n++)
            {
                if (n % 2 == 0) { continue; }
                if (n > 100) { break; }
            }

            foreach (var item in list) { yield_it(item); }
            while (a-- > 0) { do { a += 2; } while (a < 5); }

            switch (a)
            {
                case 1: goto done;
                case > 5 and < 10: break;
                default: break;
            }

            try
            {
                await Task.Delay(10);
                lock (o) { checked { a++; } }
            }
            catch (Exception ex) when (ex is not null)
            {
                throw;
            }
            finally
            {
                Dispose();
            }

        done:
            return a is not 0 ? 1 : 0;
        }

        public override string ToString() => $"{base.ToString()}";
        public void Dispose() { GC.SuppressFinalize(this); }
        public static implicit operator int(Widget<T> w) => 0;
        unsafe void Raw(int* p) { stackalloc int[4]; }
        IEnumerable<int> Gen() { yield return 1; yield break; }
    }
}
```

C++

Monday, October 26, 2026 4:42 PM

```cpp
// plain line comment
/// doc comment on a line
//! another doc comment style
/* block comment on one line */
/*
 * multi-line block comment
 * spanning several lines
 */

#include <iostream>
#include <vector>
#include "local/header.h"
#  pragma once
#define MAX_BUFFER 1024
#define SQUARE(x) ((x) * (x))
#ifdef __cplusplus
#if defined(DEBUG_BUILD)
#warning debug build
#endif
#endif

namespace app::detail {

/// A templated container
template <typename T, class Alloc = std::allocator<T>>
class Widget final : public Base {
public:
    enum class Color { Red, Green, Blue };
    struct Point { int x; int y; };
    union Value { int i; float f; };

    [[nodiscard]] static constexpr size_t Count() noexcept;

    explicit Widget(const std::string& name, uint32_t flags = 0x1F);
    virtual ~Widget() override;

    void Run(int argc, char* argv[]) const
    {
        int a = 42;
        long b = 100000L;
        unsigned c = 10u;
        double d = 3.14;
        float e = 1.5f;
        double f = 6.02e23;
        auto g = 0xDEADBEEF;
        auto h = 0b1010'0101;
        auto i = 1'000'000;

        char ch = 'x';
        char esc = '\n';
        wchar_t wide = L'w';
        const char* s1 = "plain string";
        const char* s2 = "escaped \"quote\" and \\ backslash\n";
        auto s3 = L"wide string";
        auto s4 = u8"utf-8 string";
        auto s5 = R"(raw string with "quotes" and \n inside)";

        bool t = true;
        bool u = false;
        void* p = nullptr;
        FILE* fp = NULL;

        if (a > 0 && t) {
```

```
            std::cout << "positive" << std::endl;
        } else if (a < 0 || u) {
            throw std::runtime_error("negative");
        } else {
            return;
        }

        for (int n = 0; n < MAX_BUFFER; ++n) {
            if (n % 2 == 0) { continue; }
            if (n > 100) { break; }
        }

        while (a-- > 0) { do { a += 2; } while (a < 5); }

        switch (a) {
            case 1: goto done;
            default: break;
        }

        try {
            std::vector<int> v{1, 2, 3};
            v[0] = v[1] + v[2];
            auto m = std::move(v);
            std::chrono::steady_clock::time_point tp;
        } catch (const std::exception& ex) {
            std::cerr << ex.what();
        }

        auto* w = new Widget<int>("x");
        delete w;
        auto r = static_cast<int>(d) + reinterpret_cast<intptr_t>(p);
        size_t sz = sizeof(int) + alignof(double);

    done:
        __debugbreak();
        __declspec(dllexport) int __stdcall helper();
    }

private:
    mutable std::map<std::string, std::vector<int>> items_[4];
    volatile int64_t counter_ = INT_MAX;
    thread_local static uint8_t cache_;
    time_t stamp_; ssize_t offset_; off_t pos_;
};

template <typename T>
concept Printable = requires(T t) { std::cout << t; };

typedef unsigned long long ull;
using Handle = void*;

auto Compute() -> co_task<int>
{
    co_await Delay(10);
    co_yield 1;
    co_return SQUARE(5);
}

} // namespace app::detail

int main(int argc, char** argv)
{
    app::detail::Widget<int> w("demo");
    w.Run(argc, argv);
    return EXIT_SUCCESS;
}
```

CSS

Monday, October 26, 2026 5:42 PM

```
@charset "utf-8";
@import url("other.css");
/* single line comment */
/*
  multi-line comment
*/
:root { --main-color: #06c; --gap: 8px; }
html, body { margin: 0; padding: 0; font-family: "Segoe UI", sans-serif; }
.card > .title:hover, #main a[href^="http"]::after {
  color: var(--main-color);
  background: #ff00cc url("img.png") no-repeat;
  margin: 0 auto 1.5em -2px;
  width: calc(100% - 2 * var(--gap));
  transition: all 0.3s ease-in-out;
  content: "\201C";
  opacity: .5;
  z-index: 10 !important;
}
@media (max-width: 600px) and (orientation: landscape) {
  .card { display: none; }
}
@font-face { font-family: "X"; src: url(x.woff2) format("woff2"); }
@keyframes spin { from { transform: rotate(0deg); } to { transform: rotate(360deg); } }
```

Dockerfile

Saturday, November 7, 2026 7:47 AM

```
# Comment at the start of a line
    # Indented comment

# Instructions (FROM, ARG) with variables, AS keyword, and flags
ARG VERSION=10
ARG BASE="mcr.microsoft.com/dotnet"
FROM --platform=linux/amd64 ${BASE}/sdk:$VERSION AS build
FROM scratch as final

# Lowercase and indented instructions
  workdir /src
  run echo 'single quoted string'

# Strings: single, double, and double with an escaped quote
LABEL maintainer="Jane Doe" description='A "mixed" example' note="she said \"hi\""
MAINTAINER Jane Doe

# Environment variables: braced and simple forms
ENV APP_HOME=/app PATH=$PATH:${APP_HOME}/bin
ENV EMPTY=""

# Flags (operators) on COPY and ADD
COPY --from=build --chown=1000:1000 /src/out $APP_HOME
ADD --chmod=755 https://example.com/tool.tgz /usr/local/

# Numbers: ports and other integers
EXPOSE 80 443 8080
STOPSIGNAL 9
HEALTHCHECK --interval=30s --timeout=3s --retries=3 CMD curl -f http://localhost:80/ || exit 1

# Remaining instructions
USER 1000
VOLUME ["/data", "/logs"]
SHELL ["/bin/bash", "-c"]
ONBUILD RUN echo "triggered"
ENTRYPOINT ["dotnet", "app.dll"]
CMD ["--urls", " http://+:80"]
```

Go

Monday, October 26, 2026 5:46 PM

```go
// plain line comment
/* block comment on one line */
/*
 * multi-line block comment
 */

package main

import (
	"fmt"
	"os"
	str "strings"
)

const Max = 0x7FFF_FFFF

var (
	name  string = "plain \"quoted\" string\n"
	count int
)

type Point struct {
	X, Y int
	Tag  string `json:"tag,omitempty"`
}

type Shape interface {
	Area() float64
}

type Handler func(w int, r *string) error

type Number interface {
	~int | ~int64 | ~float64
}

func Map[T, U any](xs []T, f func(T) U) []U {
	out := make([]U, 0, len(xs))
	for _, x := range xs {
		out = append(out, f(x))
	}
	return out
}

func (p *Point) Area() float64 { return float64(p.X * p.Y) }

func main() {
	// numbers
	var a int = 42
	var b int64 = 1_000_000
	var c uint8 = 0xFF
	d := 0b1010_0101
	e := 0o755
	f := 3.14e3
	g := 6.02e23
	h := 2 + 3i
	var u uintptr
	var z complex128

	// runes and strings
	r := 'x'
	esc := '\n'
```

```
	s1 := "plain \"quoted\""
	raw := `raw string
spanning lines with "quotes" and \n`

	// literals and types
	var ok bool = true
	var no bool = false
	var err error = nil
	var any1 any
	var bs []byte
	var rn rune
	var m map[string][]int
	var ch chan int
	var f32 float32
	var i8 int8
	var iota1 = iota

	// builtins
	n := len(s1) + cap(bs)
	m = make(map[string][]int)
	p := new(Point)
	bs = append(bs, 1, 2)
	copy(bs, []byte("x"))
	delete(m, "k")
	close(ch)
	println(min(1, 2), max(3, 4))
	defer fmt.Println("deferred")

	// control flow
	if a > 0 && ok {
		fmt.Println("positive")
	} else if a < 0 || no {
		panic("negative")
	} else {
		recover()
	}

	for i := 0; i < 10; i++ {
		if i%2 == 0 {
			continue
		}
		if i > 5 {
			break
		}
	}
	for range 3 {
	}
	for k, v := range m {
		_, _ = k, v
	}

	switch a {
	case 1:
		fallthrough
	case 2:
		break
	default:
	}

	select {
	case msg := <-ch:
		_ = msg
	default:
		goto done
	}

	go func() {
		ch <- 1
	}()

done:
```

```
        os.Exit(0)
}
```

Java

Monday, October 26, 2026 5:09 PM

```java
// plain line comment
/* block comment on one line */
/**
 * Javadoc comment
 * @param x something
 */

package com.example.demo;

import java.util.List;
import java.util.Map;
import static java.lang.Math.*;

@FunctionalInterface
public interface Greeter {
    String greet(String name);
}

@interface Marker {
    String value() default "x";
}

@SuppressWarnings({"unchecked", "rawtypes"})
public abstract class Widget<T extends Comparable<T>> extends Base implements Greeter, AutoCloseable {

    public enum Kind { SMALL, LARGE }
    public record Point(int x, int y) { }
    sealed interface Shape permits Circle, Square { }

    private static final int MAX = 0x7FFF_FFFF;
    private final String name = "plain \"quoted\" string\n";
    protected transient volatile boolean flag;

    @Override
    public String greet(String name) {
        // numbers
        int a = 42;
        long b = 100_000L;
        float c = 1.5f;
        double d = 6.02e23;
        double e = 3.14;
        int f = 0xDEADBEEF;
        int g = 0b1010_0101;

        // chars and strings
        char ch = 'x';
        char esc = '\n';
        String s1 = "plain";
        String s2 = """
            text block
            with "quotes" inside
            """;

        // literals and types
        Object o = null;
        boolean t = true, u = false;
        var list = new java.util.ArrayList<String>();
        byte by = 1; short sh = 2;
        Integer boxed = 5; Optional<String> opt = Optional.empty();
        Map<String, List<Integer>> map = new java.util.HashMap<>();

        // control flow
        if (a > 0 && t) {
            System.out.println("positive");
        } else if (a < 0 || u) {
            throw new IllegalStateException("negative");
        }
```

```
        for (int n = 0; n < MAX; n++) {
            if (n % 2 == 0) { continue; }
            if (n > 100) { break; }
        }

        for (String item : list) { assert item != null; }
        while (a-- > 0) { do { a += 2; } while (a < 5); }

        switch (a) {
            case 1 -> System.out.println("one");
            case 2 -> { break; }
            default -> System.out.println("other");
        }

        try {
            synchronized (this) { a++; }
        } catch (RuntimeException | Error ex) {
            throw ex;
        } finally {
            close();
        }

        Object r = switch (a) { case 1 -> "x"; default -> { yield "y"; } };
        return (o instanceof String str) ? str : name;
    }

    @Override public void close() { }
    public static native void nativeCall();
    strictfp double precise() { return 1.0; }
}
```

JavaScript

Monday, October 26, 2026 5:11 PM

```
// plain line comment
/* block comment on one line */
/**
 * JSDoc comment
 * @param {number} x
 */

import { readFile } from 'fs';
import * as path from "path";
export default class Widget extends Base {
  static count = 0n;
  #secret = 42;

  @observable name = 'plain';

  constructor(name) {
    super();
    this.name = name;
  }

  static async create(x) {
    // numbers
    const a = 42;
    const b = 1_000_000;
    const c = 3.14;
    const d = 6.02e23;
    const e = 0xDEADBEEF;
    const f = 0b1010_0101;
    const g = 0o755;
    const h = 123456789n;

    // strings
    const s1 = 'single \'quoted\' string';
    const s2 = "double \"quoted\" string";
    const s3 = `template ${a + b} string
spanning lines with ${`nested ${c}`}`;

    // regular expressions vs. division
    const re = /ab+c[/]\//gi;
    const ratio = a / b / c;
    const parts = 'a,b'.split(/,\s*/);

    // literals
    let u = undefined, n = NaN, i = Infinity, z = null, y = true, w = false;
    var old = typeof z === 'object';

    // types / globals
    const list = new Array(3);
    const map = new Map();
    const set = new Set([1, 2]);
    const p = Promise.resolve(new Date());
    const obj = Object.freeze({ a: 1 });
    const text = String(a) + Number('1') + Boolean(0);
    const sym = Symbol('x'), json = JSON.stringify(Math.max(1, 2));

    // control flow
    if (a > 0 && y) {
      console.log('positive');
    } else if (a < 0 || w) {
      throw new TypeError('negative');
    }

    for (let k = 0; k < 10; k++) {
```

```
      if (k % 2 === 0) { continue; }
      if (k > 5) { break; }
    }
    for (const key in obj) { delete obj[key]; }
    for (const v of list) { void v; }
    while (a-- > 0) { do { a += 2; } while (a < 5); }

    switch (a) {
      case 1: break;
      default: debugger;
    }

    try {
      await p;
      const r = await fetch(`/api/${a}`);
      return yield_all(r);
    } catch (err) {
      throw err;
    } finally {
      with (obj) { }
    }
  }

  get value() { return this.#secret; }
  set value(v) { this.#secret = v; }
  *gen() { yield 1; yield* [2, 3]; }
}

function legacy(arg) { return arg instanceof Widget ? arg : null; }
const arrow = async (x) => x ** 2;
export { legacy, arrow };
```

JSON

Monday, October 26, 2026 4:52 PM

```jsonc
// line comment (JSONC)
/* block comment on one line */
{
  "string": "plain \"quoted\" string",
  "integer": 42,
  "negative": -17,
  "float": 3.14,
  "exponent": 6.02e23,
  "smallExp": 1.5E-3,
  "signedExp": 2e+10,
  "true": true,
  "false": false,
  "null": null,
  "array": [1, 2.5, "three", true, null, -4e2],
  "nested": {
    "key": "value",
    "list": [
      { "a": 1 },
      { "b": [ ] }
    ],
    "empty": {}
  },
  "url": "https://example.com/path?x=1&y=2",
  "unicode": "café \n tab\t"
}
```

HTML

Monday, October 26, 2026 4:58 PM

```
<!DOCTYPE html>
<html>
<body>

<h3 style="color:Tomato;">Hello World</h3>

<p style="color:DodgerBlue;">Lorem ipsum dolor sit amet, consectetuer adipiscing elit, sed diam nonummy nibh euismod tincidunt ut laoreet dolore magna aliquam erat volutpat.</p>

<p style="color:MediumSeaGreen;">Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat.</p>

</body>
</html>
```

Onetastic

Monday, October 26, 2026 5:32 PM

```
// plain line comment
/* block comment on one line */
/*
   multi-line block comment
*/

// variables and numbers
$count = 42;
$ratio = 3.14;
$name = "plain \"quoted\" string";
$page = Page_GetCurrent();
$title = $page.Title;
$items = [];
$flag = true;
$other = false;

// function calls
$section = Section_GetCurrent();
$notebook = Notebook_GetCurrent();
Page_SetTitle($page, "New Title");
Window_GetCurrent();
String_Replace($name, "a", "b");
Math_Floor(3.7);
Array_Add($items, 1);

// control flow
if ($count > 0 && $flag)
{
    Debug_Print("positive");
}
else if ($count < 0 || $other)
{
    return;
}
else
{
    Debug_Print("zero");
}

for ($i = 0; $i < 10; $i++)
{
    if ($i % 2 == 0) { continue; }
    if ($i > 5) { break; }
}

foreach ($item in $items)
{
    Debug_Print($item);
}

while ($count > 0)
{
    $count = $count - 1;
}

switch ($count)
{
    case 1:
        break;
    default:
        break;
}

function Helper(byref $x, $y)
```

```
{
    $x = $x + $y;
    return $x;
}
```

Pascal

Monday, October 26, 2026 5:30 PM

```
// plain line comment
{ brace comment on one line }
{
  multi-line brace comment
}
(* paren-star comment on one line *)
(*
  multi-line paren-star comment
*)

{$mode objfpc}{$H+}
{$IFDEF DEBUG}
{$DEFINE TRACE}
{$ENDIF}

program Sample;

uses SysUtils, Classes;

const
  Max = 100;
  Mask = $FF;
  Pi2 = 6.28;
  Big = 1.5e10;
  Greeting = 'it''s a "string"';
  Tab = #9;
  Newline = #13#10;
  Smile = #$263A;

type
  TColor = (Red, Green, Blue);
  TPoint = record
    X, Y: Integer;
  end;
  TWidget = class(TObject)
  private
    FName: string;
    FCount: Longint;
  public
    constructor Create(const AName: string);
    destructor Destroy; override;
    function Area(W, H: Real): Double;
    procedure Run;
  end;

var
  a, b: Integer;
  c: Byte;
  d: Word;
  e: Boolean;
  f: Char;
  g: Real;
  h: Single;
  i64: Int64;
  names: array[0..9] of string;
  p: Pointer;

constructor TWidget.Create(const AName: string);
begin
  inherited Create;
  FName := AName;
end;

function TWidget.Area(W, H: Real): Double;
```

```
begin
  Result := W * H;
end;

procedure TWidget.Run;
var
  k: Integer;
begin
  a := 42;
  g := 3.14;
  e := True;
  p := nil;
  WriteLn('Count: ', a, ' ', Format('%d', [a]));

  if (a > 0) and e then
    WriteLn('positive')
  else if (a < 0) or not e then
    Halt(1)
  else
    Exit;

  for k := 0 to 9 do
  begin
    if k mod 2 = 0 then Continue;
    if k > 5 then Break;
  end;
  for k := 9 downto 0 do Inc(a);

  while a > 0 do Dec(a);
  repeat
    a := a + 2;
  until a >= 5;

  case a of
    1: WriteLn('one');
    2..9: WriteLn('few');
  else
    WriteLn('many');
  end;

  try
    a := a div 0;
  except
    on E: Exception do WriteLn(E.Message);
  end;

  with Self do FCount := FCount shl 2;
end;

begin
  with TWidget.Create('x') do
  try
    Run;
  finally
    Free;
  end;
end.
```

PlantUML

Monday, October 26, 2026 5:44 PM

```
@startuml
' single line comment
/' block comment on one line '/
/'
  multi-line block comment
'/
title Sample Diagram
skinparam monochrome true
!include common.puml

actor User
participant "Web Server" as Web
database DB
class Order {
  +id : int
  -total : decimal
  +place() : void
}
interface Payable
enum Status

User -> Web : request
Web --> DB : query
DB ->> Web : rows
Web -x User : error
User <- Web : response

note left of User : a note
note over Web, DB
  multi-line note
end note

alt success
  Web -> User : ok
else failure
  Web -> User : fail
end
loop 3 times
  User -> Web : retry
end
Order --|> Payable
Order "1" *-- "many" Item : contains
@enduml
```

PowerShell

Monday, October 26, 2026 5:24 PM

```powershell
# plain line comment
<# block comment on one line #>
<#
.SYNOPSIS
    multi-line block comment
#>

[CmdletBinding()]
param(
    [Parameter(Mandatory = $true)]
    [string]$Name,
    [int]$Count = 10
)

function Get-Widget {
    [CmdletBinding()]
    param([Parameter(ValueFromPipeline)][string]$Item)
    begin { $list = @() }
    process { $list += $Item }
    end { return $list }
}

# numbers
$a = 42
$b = 3.14
$c = 6.02e23
$d = 0xDEADBEEF
$e = 1kb + 2MB + 3gb
$f = 100L

# strings
$s1 = 'single ''quoted'' string'
$s2 = "double `"quoted`" with $a and $($b + 1) and ${c}"
$s3 = @"
here-string with "quotes" and $a
"@
$s4 = @'
literal here-string with 'quotes' and $a
'@

# variables and types
$t = $true; $u = $false; $n = $null
[int]$typed = 5
[string[]]$names = 'x', 'y'
$env:PATH; $global:Setting = 1; $script:Counter = 0; $_; $?; $$
$arr = @(1, 2, 3); $hash = @{ Key = 'value'; Other = 2 }
[System.IO.Path]::Combine('a', 'b')
[Math]::Max(1, 2)

# operators
if ($a -eq 42 -and $b -gt 3 -or $c -ne 0) { Write-Host 'match' }
if ($Name -like '*x*' -and $Name -notmatch '^\d') { Write-Output "ok" }
$x = $a + $b - $c * $d / $e % 2
$a += 1; $b -= 1; $c *= 2; $d /= 2; $a++; $b--
Get-ChildItem -Path . | Where-Object { $_.Length -gt 100 } > out.txt 2>&1

# control flow
if ($a -gt 0) {
    Write-Host 'positive'
} elseif ($a -lt 0) {
    throw 'negative'
} else {
    exit 1
}
```

```
for ($i = 0; $i -lt 10; $i++) {
    if ($i % 2 -eq 0) { continue }
    if ($i -gt 5) { break }
}
foreach ($item in $arr) { Write-Output $item }
while ($a -gt 0) { $a-- }
do { $a++ } until ($a -ge 5)

switch ($a) {
    1 { 'one' }
    default { 'other' }
}

try {
    Get-Item 'nope' -ErrorAction Stop
} catch {
    Write-Error $_
} finally {
    Remove-Item temp.txt -Force
}
```

PHP

Monday, October 26, 2026 5:28 PM

```php
<?php
// plain line comment
# hash line comment
/* block comment on one line */
/**
 * Doc comment
 */

declare(strict_types=1);

namespace App\Models;

use InvalidArgumentException;

#[Attribute(Attribute::TARGET_CLASS)]
final class Marker {}

interface Shape { public function area(): float; }
trait Loggable { protected function log(string $m): void {} }
enum Status: string { case Active = 'a'; case Off = 'o'; }

#[Marker]
abstract class Widget extends Base implements Shape
{
    use Loggable;

    public const MAX = 0x7FFF_FFFF;
    private static int $count = 0;
    protected ?string $name = null;
    public readonly array $items;

    public function __construct(string $name, int ...$rest)
    {
        $this->name = $name;
        echo __CLASS__ . ' ' . __FUNCTION__ . ' ' . __LINE__;
    }

    public function run(): mixed
    {
        // numbers
        $a = 42;
        $b = 1_000_000;
        $c = 3.14;
        $d = 6.02e23;
        $e = 0xDEADBEEF;
        $f = 0b1010_0101;
        $g = 0o755;
        $h = 0755;

        // strings
        $s1 = 'single \'quoted\'';
        $s2 = "double \"quoted\" with $a and {$b}";

        // literals and types
        $t = true; $u = FALSE; $n = null;
        $arr = array(1, 2); $obj = new \stdClass();
        $i = (int) $c; $fl = (float) $a; $bo = (bool) $a; $st = (string) $a;
        $cb = fn(int $x): int => $x * 2;

        // control flow
        if ($a > 0 && $t) {
            echo 'positive';
```

```php
        } elseif ($a < 0 || $u) {
            throw new InvalidArgumentException('negative');
        } else {
            return null;
        }

        for ($k = 0; $k < 10; $k++) {
            if ($k % 2 === 0) { continue; }
            if ($k > 5) { break; }
        }
        foreach ($arr as $key => $v) { print $v; }
        while ($a-- > 0) { do { $a += 2; } while ($a < 5); }

        switch ($a) {
            case 1: break;
            default: break;
        }

        $r = match (true) { $a > 1 => 'big', default => 'small' };

        try {
            yield $a;
            include 'file.php';
            require_once 'other.php';
        } catch (\Exception $ex) {
            die('error');
        } finally {
            unset($arr);
        }
        return isset($obj) && !empty($arr) ? clone $obj : exit(1);
    }
}
?>
```

Python

Monday, October 26, 2026 5:27 PM

```python
#!/usr/bin/env python3
# plain line comment
"""Module docstring
spanning several lines"""
'''Another docstring'''

import os
from typing import List, Optional


@dataclass(frozen=True)
class Widget(Base):
    """Class docstring"""

    count: int = 0

    def __init__(self, name: str, *args, **kwargs) -> None:
        super().__init__()
        self.name = name

    @staticmethod
    async def create(x: float = 3.14) -> "Widget":
        # numbers
        a = 42
        b = 1_000_000
        c = 3.14
        d = 6.02e23
        e = 0xDEADBEEF
        f = 0b1010_0101
        g = 0o755
        h = 2 + 3j

        # strings
        s1 = 'single \'quoted\''
        s2 = "double \"quoted\""
        s3 = r"raw \n string"
        s4 = f"formatted {a} and {b!r}"
        s5 = b"bytes"
        s6 = """triple quoted
        string"""

        # literals and builtin types
        t, f2, n = True, False, None
        xs: list = [1, 2]
        ds: dict = {"k": 1}
        st: set = {1}
        tp: tuple = (1,)
        fl: float = float("1.5")
        by: bytes = bytes(3)

        # control flow
        if a > 0 and t:
            print("positive")
        elif a < 0 or f2:
            raise ValueError("negative")
        else:
            pass

        for i in range(10):
            if i % 2 == 0:
                continue
            if i > 5:
                break
        -
```

```
else:
    pass

while a > 0:
    a -= 1

try:
    with open("f") as fh:
        data = fh.read()
except (IOError, OSError) as ex:
    raise
finally:
    del xs

match a:
    case 1:
        return None
    case _:
        yield a

lam = lambda q: q ** 2
return await lam(2) if a is not None and a not in xs else global_fn()
```

Registry

Monday, October 26, 2026 5:26 PM

```
Windows Registry Editor Version 5.00

; plain comment line

[HKEY_CURRENT_USER\Software\Sample]
"StringValue"="plain \"quoted\" string"
"PathValue"="C:\\Program Files\\Sample"
"DwordValue"=dword:0000002a
"DwordMax"=dword:ffffffff
"BinaryValue"=hex:de,ad,be,ef,00,01
"QwordValue"=hex(b):2a,00,00,00,00,00,00,00
"ExpandValue"=hex(2):25,00,50,00,41,00,54,00,48,00,25,00,00,00
"MultiValue"=hex(7):61,00,00,00,62,00,00,00,00,00
@="default value"

[HKEY_LOCAL_MACHINE\SOFTWARE\Sample\Sub Key]
"Enabled"=dword:00000001
"Name"="Sample 42"

[-HKEY_CURRENT_USER\Software\Obsolete]

[HKEY_CURRENT_USER\Software\Sample\Gone]
"RemoveMe"=-



[HKEY_CLASSES_ROOT\onemore]
@="URL:OneMore Protocol Handler"
"URL Protocol"=""

[HKEY_CLASSES_ROOT\onemore\shell\open\command]
@="\"C:\\Program Files\\River\\OneMoreAddIn\\OneMoreProtocolHandler.exe\" %1 %2 %3 %4 %5"
"Logging"="voicewarmupx"
"foo"=hex(2):abc123
"fox"=dword:0004

-[HKEY_LOCAL_MACHINE\SOFTWARE\Classes]
```

R

Monday, October 26, 2026 5:30 PM

```r
# plain line comment

library(dplyr)

# numbers
a <- 42
b <- 42L
c <- 3.14
d <- 6.02e23
e <- 0xDEADBEEF
f <- 1e-3

# strings
s1 <- "double \"quoted\" string"
s2 <- 'single \'quoted\' string'

# literals
t <- TRUE; u <- FALSE; n <- NULL
miss <- NA; inf <- Inf; nan <- NaN
mi <- NA_integer_; mr <- NA_real_; mc <- NA_character_

# functions and calls
square <- function(x, ...) {
  x^2
}
v <- c(1, 2, 3)
m <- matrix(0, nrow = 2, ncol = 2)
df <- data.frame(x = 1:3, y = c("a", "b", "c"))
paste("a", "b", sep = "-")
print(length(v))
result <- sapply(v, function(i) i * 2)
df %>% filter(x > 1) %>% summarise(n = n())

# control flow
if (a > 0 && t) {
  print("positive")
} else if (a < 0 || u) {
  stop("negative")
} else {
  return(NULL)
}

for (i in 1:10) {
  if (i %% 2 == 0) next
  if (i > 5) break
}

while (a > 0) {
  a <- a - 1
}

repeat {
  a <- a + 1
  if (a >= 5) break
}

switch("b", a = "first", b = "second")
tryCatch(stop("boom"), error = function(e) conditionMessage(e), finally = cat("done\n"))
```

Regex

Tuesday, October 27, 2026 4:23 PM

```
(?# 1. Inline comment group)
(?# a comment group)abc(?#another one)def

(?# 2a. Character classes: delimiters, literals and ranges)
[abc]
[^abc]
[a-zA-Z0-9_]
[-+*/.,;:!]

(?# 2b. POSIX classes inside a class)
[[:alpha:][:digit:]]
[[:^space:]]

(?# 2c. Shorthand and property classes inside a class)
[\d\w\s\h]
[\D\W\S\H]
[\p{L}\P{Nd}]

(?# 2d. Escapes inside a class)
[\n\r\t]
[\x41\x7eéۓ€]
[\cA\cZ]
[\0\12\377]
[\]\[\-\\\^]

(?# 2e. Mixed class contents)
[\w.+-]
[^\x00-\x1f\x7f]
[a-z\d_[:blank:]\]]

(?# 2f. Class subtraction leaves a stray closing bracket)
[a-z-[aeiou]]

(?# 3a. Named groups: angle, Python and balancing forms)
(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})
(?P<user>\w+)@(?P<host>[\w.]+)
(?<open-close>\))
(?<-open>\(

(?# 3b. Named groups with quotes)
(?'word'\w+)\s+(?'num'\d+)
(?'a-b'x)

(?# 3c. Lookaround, atomic and conditional openers)
(?=ahead)
(?!not-ahead)
(?<=behind)
(?<!not-behind)
(?>atomic)
(?(open)then|else)
(?(?=\d)\d+|[a-z]+)

(?# 3d. Non-capturing groups)
(?:abc|def)+

(?# 3e. Inline options)
(?i)case-insensitive
(?imnsx)all-flags
(?i-s)on-and-off
(?-i)off-only
(?i:scoped)
(?i-s:scoped on and off)
```

```
(?-ix:scoped off)

(?# 3f. Plain capture groups)
(a)(b(c))

(?# 4a. Backreferences)
(\w)\1
(\w+)\s\1\s\2
(?<w>\w+)\k<w>
(?'w'\w+)\k'w'
(?<w>\w+)\k{w}

(?# 4b. Anchors and boundaries)
\bword\b
\Bmid\B
\Astart
end\z
end\Z
\Gcontinue
foo\Kbar

(?# 4c. Shorthand and property classes outside a class)
\d\D\w\W\s\S
\h\H\R\N\X
\p{L}\P{Lu}\p{IsGreek}
\pL\PL\pN

(?# 4d. Other escapes)
\n\r\t\f\v\a\e
\x41\x7E\x{1F600}
é€
\cA\cZ
\0\01\012
\.\(\)\[\]\{\}\*\+\?\|\^\$\\\/

(?# 5a. Counted quantifiers)
a{3}
a{3,}
a{3,5}
a{,5}

(?# 5b. Star, plus, question: lazy and possessive suffixes)
a*  a+  a?
a*? a+? a??
a*+ a++ a?+
<.+?>
"[^"]*+"

(?# 6a. Line anchors)
^start
end$
^$

(?# 6b. Alternation and any-character)
cat|dog|bird
a.b.c
.*

(?# 7. Realistic combined patterns)
(?i)^[\w.+-]+@(?:[a-z\d-]+\.)+[a-z]{2,}$
\b(?<y>\d{4})-(?<m>0[1-9]|1[0-2])-(?<d>[0-2]\d|3[01])\b(?=\s|$)
\b(?<word>\w+)\b(?#a word)\s+\k<word>\b
"(?:[^"\\]|\\.)*+"
^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\w\s]).{8,64}$
^[\p{L}_][\p{L}\p{Nd}_]*$
\((?:[^()]|(?<open>\()|(?<-open>\)))*(?(open)(?!))\)
```

Rust

Monday, October 26, 2026 5:10 PM

```rust
// plain line comment
/* block comment on one line */
/*
 * multi-line block comment
 */
/// outer doc comment
//! inner doc comment

#![allow(dead_code, unused_variables)]

use std::collections::{BTreeMap, HashMap, HashSet};
use std::rc::Rc;
use std::sync::Arc;

pub mod util {
    pub const MAX: u32 = 0x7FFF_FFFF;
    pub static NAME: &str = "plain \"quoted\" string\n";
}

#[derive(Debug, Clone, PartialEq)]
pub struct Point<'a> {
    pub x: i32,
    pub y: i32,
    name: &'a str,
}

#[derive(Debug)]
pub enum Shape {
    Circle(f64),
    Rect { w: f64, h: f64 },
    Empty,
}

pub union IntOrFloat {
    i: u32,
    f: f32,
}

pub type Result2<T> = std::result::Result<T, Box<dyn std::error::Error>>;

pub trait Area {
    fn area(&self) -> f64;
}

impl<'a> Area for Point<'a> {
    fn area(&self) -> f64 {
        (self.x * self.y) as f64
    }
}

impl<'a> Point<'a> {
    pub fn new(name: &'a str) -> Self {
        Self { x: 0, y: 0, name }
    }
}

#[cfg(test)]
mod tests {
    #[test]
    fn it_works() {
        assert_eq!(2 + 2, 4);
    }
```

```rust
}

async fn fetch(url: &str) -> Result<String, std::io::Error> {
    let body = reqwest::get(url).await?.text().await?;
    Ok(body)
}

unsafe extern "C" fn raw(p: *const u8, q: *mut u8) {}

fn main() {
    // numbers
    let a = 42;
    let b = 1_000_000i64;
    let c = 0xFF_u8;
    let d = 0b1010_0101;
    let e = 0o755;
    let f = 3.14f32;
    let g = 6.02e23_f64;
    let h = 1e-3;
    let sz: usize = 10;
    let big: u128 = 1;

    // chars and strings
    let ch = 'x';
    let esc = '\n';
    let uni = '\u{1F600}';
    let byte = b'a';
    let s1 = "plain \"quoted\"";
    let s2 = r"raw string";
    let s3 = r#"raw "quoted" string"#;
    let s4 = b"byte string";
    let multi = "spans
lines";

    // literals and types
    let t: bool = true;
    let n: Option<i32> = None;
    let v: Vec<i32> = vec![1, 2, 3];
    let st: String = String::from("x");
    let m: HashMap<String, i32> = HashMap::new();
    let b2: Box<i32> = Box::new(5);
    let rc: Rc<str> = Rc::from("x");
    let stat: &'static str = "static";
    let tup: (i8, i16, u16, isize, f64, char, bool) = (1, 2, 3, 4, 5.0, 'c', true);
    let path = std::collections::HashMap::<i32, i32>::new();

    // macros
    println!("{} {}", a, b);
    let s = format!("{:?}", v);
    assert!(a > 0, "positive");
    panic!("boom");

    // control flow
    if a > 0 && t {
        println!("positive");
    } else if a < 0 {
        return;
    } else {
        todo!();
    }

    for i in 0..10 {
        if i % 2 == 0 { continue; }
        if i > 5 { break; }
    }
    while let Some(x) = Some(1) { break; }

    let mut count = 0;
    loop {
        count += 1;
        if count > 3 { break; }
    }
```

```
    }

    match a {
        1 | 2 => println!("small"),
        3..=9 => {}
        x if x > 100 => {}
        _ => {}
    }

    let closure = move |x: i32| -> i32 { x + 1 };
    let r = unsafe { *(&a as *const i32) };
    let _ = (n, st, m, b2, rc, stat, tup, path, s, closure, r, ch, esc, uni, byte);
}
```

Scala

Monday, October 26, 2026 5:34 PM

```scala
// plain line comment
/* block comment on one line */
/**
 * Scaladoc comment
 * @param x something
 */

package com.example.demo

import scala.collection.mutable
import scala.concurrent.Future

@deprecated("old", "1.0")
trait Shape { def area: Double }

sealed abstract class Base
case class Point(x: Int, y: Int) extends Base
object Util { val Max: Int = 0x7FFF_FFFF }
enum Color { case Red, Green, Blue }
type Pair[A] = (A, A)
given Ordering[Int] = Ordering.Int

@main
class Widget[T](name: String) extends Base with Shape {

  override def area: Double = 3.14

  def run(args: Array[String]): Unit = {
    // numbers
    val a = 42
    val b = 100L
    val c = 3.14
    val d = 6.02e23
    val e = 1.5f
    val f = 0xDEADBEEF

    // strings
    val s1 = "plain \"quoted\" string"
    val ch = 'x'
    val s2 = """triple quoted
      "string" """
    val s3 = s"interpolated $a and ${b + 1}"

    // literals and types
    val t: Boolean = true
    val n: Any = null
    val o: Option[String] = None
    val l: List[Int] = List(1, 2, 3)
    val m: Map[String, Int] = Map("k" -> 1)
    val sq: Seq[Long] = Seq(1L)
    val r: Either[String, Int] = Right(1)
    val fu: Future[Unit] = Future.successful(())
    var counter: Short = 0

    // control flow
    if (a > 0 && t) {
      println("positive")
    } else if (a < 0) {
      throw new IllegalStateException("negative")
    } else {
      return
    }
```

```
    for (i <- 1 to 10) {
      if (i % 2 == 0) println(i)
    }
    for { x <- l; y <- l } yield x * y

    while (counter < 5) { counter = (counter + 1).toShort }

    a match {
      case 1 => println("one")
      case _ => println("other")
    }

    try {
      lazy val z = a / 0
    } catch {
      case e: ArithmeticException => println(e)
    } finally {
      println("done")
    }
  }

  private implicit def conv(x: Int): String = x.toString
  protected final def fin(): Unit = {}
}
```

SQL

Monday, October 26, 2026 4:45 PM

```sql
-- plain line comment
/* block comment on one line */
/*
   multi-line block comment
*/

USE SampleDb;
GO

DECLARE @Count INT = 42, @Name NVARCHAR(50) = N'Widget', @Price DECIMAL(10, 2) = 19.99;
DECLARE @Hex BINARY(4) = 0xDEADBEEF, @Ratio FLOAT = 6.02e23, @Id UNIQUEIDENTIFIER = NEWID();
DECLARE @When DATETIME = GETDATE(), @Flag BIT = 1, @Money MONEY = 5;

CREATE TABLE [dbo].[Widgets] (
    Id          INT IDENTITY(1, 1) PRIMARY KEY,
    Name        VARCHAR(100) NOT NULL,
    Notes       TEXT NULL,
    Price       NUMERIC(8, 2) DEFAULT 0,
    CreatedOn   DATETIME2 DEFAULT CURRENT_TIMESTAMP,
    Token       VARBINARY(16),
    Doc         XML
);

SELECT TOP 10 w.Id, w.Name AS "Display Name", COUNT(*) AS Total, SUM(w.Price) AS Cost
FROM dbo.Widgets AS w
INNER JOIN dbo.Orders o ON o.WidgetId = w.Id
LEFT JOIN dbo.Owners ow ON ow.Id = o.OwnerId
WHERE w.Name LIKE 'Wid%' AND w.Price BETWEEN 1.5 AND 100 OR w.Id IN (1, 2, 3)
GROUP BY w.Id, w.Name
HAVING COUNT(*) > 1
ORDER BY w.Name DESC;

INSERT INTO dbo.Widgets (Name, Price) VALUES ('It''s a widget', 9.99);
UPDATE dbo.Widgets SET Price = Price * 1.1 WHERE Id = @Count;
DELETE FROM dbo.Widgets WHERE CreatedOn < DATEADD(DAY, -30, GETDATE());

IF @@ROWCOUNT > 0
BEGIN
    PRINT 'changed ' + CAST(@@ROWCOUNT AS VARCHAR(10));
END
ELSE
BEGIN
    RAISERROR('none', 16, 1);
END

WHILE @Count > 0
BEGIN
    SET @Count = @Count - 1;
    IF @Count = 5 BREAK;
    CONTINUE;
END

BEGIN TRY
    SELECT CASE WHEN @Flag = 1 THEN 'yes' ELSE 'no' END, ISNULL(@Name, 'x'), COALESCE(NULL, 1),
LEN(@Name), UPPER(@Name);
END TRY
BEGIN CATCH
    SELECT ERROR_MESSAGE(), @@ERROR;
END CATCH;

CREATE PROCEDURE dbo.GetWidget @Id INT AS
BEGIN
    SET NOCOUNT ON;
    SELECT * FROM dbo.Widgets WHERE Id = @Id;
    RETURN 0;
```

```
END;
```

TypeScript

Monday, October 26, 2026 5:10 PM

```typescript
// plain line comment
/* block comment on one line */
/**
 * TSDoc comment
 * @param x something
 */

import type { Foo } from './foo';
import { Component, Input } from '@angular/core';

declare module "ambient" { }
declare global { interface Window { app: unknown; } }

export namespace Geometry {
  export interface Shape { area(): number; readonly name: string; }
  export type Point = { x: number; y: number };
}

export enum Kind { Small, Large = 10 }
export const enum Flags { None = 0, A = 1 << 0 }

type Alias<T> = Map<string, Array<T>> | undefined;
type Mapped<T> = { readonly [K in keyof T]?: T[K] };
type Fn = (x: number, ...rest: string[]) => void | never;

@Component({ selector: 'app-widget' })
export abstract class Widget<T extends object = {}> extends Base implements Geometry.Shape {
  @Input() label: string = 'plain';
  private readonly secret: number = 0xFF;
  protected static count: bigint = 0n;
  public optional?: boolean;
  declare inferred: unknown;

  constructor(public name: string, private id?: number) {
    super();
  }

  abstract area(): number;

  override toString(): string {
    // numbers
    const a: number = 42;
    const b: number = 1_000_000;
    const c: number = 3.14e3;
    const d: number = 0b1010;
    const big: bigint = 123456789n;

    // strings
    const s1: string = 'single \'quoted\'';
    const s2: string = "double \"quoted\"";
    const s3: string = `template ${a + b}
spanning lines`;

    // regex vs. division
    const re: RegExp = /ab+c[/]\//gi;
    const ratio = a / b / c;

    // literals and types
    let u: undefined = undefined;
    let n: null = null;
    let t: boolean = true, f: boolean = false;
    let any1: any = {};
```

```
    let unk: unknown = any1;
    let nev: never;
    let sym: symbol = Symbol('x');
    let obj: object = {};
    let list: Array<string> = new Array();
    let tuple: [string, number] = ['a', 1];
    const p: Promise<void> = Promise.resolve();

    // type operations
    const cast = unk as string;
    const sat = { a: 1 } satisfies Record<string, number>;
    if (typeof unk === 'string' && unk instanceof Object) { }
    function isStr(x: unknown): x is string { return typeof x === 'string'; }
    function assert(x: unknown): asserts x { }

    // control flow
    if (a > 0 && t) {
      console.log('positive');
    } else if (a < 0 || f) {
      throw new Error('negative');
    }

    for (let k = 0; k < 10; k++) {
      if (k % 2 === 0) { continue; }
      if (k > 5) { break; }
    }
    for (const key in obj) { }
    for (const v of list) { }
    while (a-- > 0) { do { a += 2; } while (a < 5); }

    switch (a) {
      case 1: break;
      default: break;
    }

    try {
      return s1;
    } catch (err) {
      throw err;
    } finally {
      debugger;
    }
  }
}

export function generic<K extends keyof T, T>(obj: T, key: K): T[K] {
  return obj[key];
}

export default Widget;
```

Verilog

Monday, October 26, 2026 5:35 PM

```
// plain line comment
/* block comment on one line */
/*
 * multi-line block comment
 */

`timescale 1ns / 1ps
`define WIDTH 8
`include "defs.vh"
`ifdef SIMULATION
`default_nettype none
`endif

module counter #(parameter WIDTH = 8, parameter [3:0] INIT = 4'b1010) (
    input  wire             clk,
    input  wire             rst_n,
    input  wire             en,
    output reg  [WIDTH-1:0] count,
    output wire             done,
    inout  wire             bus
);

    localparam MAX = 8'hFF;
    localparam BIG = 32'd1_000_000;
    localparam OCT = 6'o77;
    localparam REAL_VAL = 3.14;
    localparam EXP_VAL = 1.5e-3;

    reg  [7:0] mem [0:15];
    wire       tick;
    integer    i;
    logic      flag;
    bit        b;
    byte       by;
    int        n;
    longint    ln;
    shortint   si;
    real       r;
    string     s = "plain \"quoted\" string\n";
    time       t0;

    assign done = (count == MAX) ? 1'b1 : 1'b0;
    assign tick = en & ~rst_n | 1'bz ^ 1'bx;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            count <= {WIDTH{1'b0}};
        end else if (en) begin
            count <= count + 1;
        end else begin
            count <= count;
        end
    end

    always_comb begin
        case (count)
            8'h00:   flag = 1'b1;
            8'h01:   flag = 1'b0;
            default: flag = 1'bx;
        endcase
    end

    initial begin
        $display("count = %d at %t", count, $time);
```

```
        $monitor("%b", count);
        for (i = 0; i < 16; i = i + 1) begin
            mem[i] = i;
        end
        repeat (4) @(posedge clk);
        while (count != 0) #10 count = count - 1;
        forever begin
            #5 clk_gen = ~clk_gen;
            if (count > 100) disable gen;
        end
        $finish;
    end

    function automatic [7:0] twice(input [7:0] x);
        twice = x << 1;
    endfunction

    task automatic wait_cycles(input integer cycles);
        repeat (cycles) @(posedge clk);
    endtask

endmodule
```

Visual Basic

Monday, October 26, 2026 5:37 PM

```
''' <summary>XML doc comment</summary>
' plain line comment
REM remark comment

#Const DEBUG = True
#Region "Imports"
Imports System
Imports System.Collections.Generic
#End Region
#If DEBUG Then
#Else
#End If

Option Explicit On
Option Strict On

<Serializable>
Public Class Widget
    Inherits Base
    Implements IDisposable

    <Obsolete("old")>
    Private Const Max As Integer = 100
    Private ReadOnly _name As String = "plain ""quoted"" string"
    Public Property Count As Long

    Public Enum Color
        Red
        Green
    End Enum

    Public Structure Point
        Public X As Integer
        Public Y As Integer
    End Structure

    Public Interface IThing
        Sub Run()
    End Interface

    Public Module Helpers
    End Module

    Public Sub New(name As String)
        MyBase.New()
        _name = name
    End Sub

    Public Async Function RunAsync(ByVal x As Double, Optional ByRef y As Single = 1.5F) As Task(Of
Integer)
        ' numbers
        Dim a As Integer = 42
        Dim b As Long = 100000L
        Dim c As Double = 3.14
        Dim d As Double = 6.02E23
        Dim e As Integer = &HFF
        Dim f As Decimal = 19.99D
        Dim g As Byte = 1
        Dim h As Short = 2
        Dim i As UInteger = 3UI

        ' strings and literals
        Dim s As String = "it's ""quoted"""
        Dim ch As Char = "x"c
        Dim dt As Date = #1/1/2020#
        Dim o As Object = Nothing
        Dim t As Boolean = True
```

```vb
        Dim u As Boolean = False
        Dim lst As New List(Of String)

        ' control flow
        If a > 0 AndAlso t Then
            Console.WriteLine("positive")
        ElseIf a < 0 OrElse u Then
            Throw New InvalidOperationException("negative")
        Else
            Return 0
        End If

        For i2 As Integer = 0 To 9
            If i2 Mod 2 = 0 Then Continue For
            If i2 > 5 Then Exit For
        Next

        For Each item As String In lst
            Console.WriteLine(item)
        Next

        While a > 0
            a -= 1
        End While

        Do
            a += 2
        Loop Until a >= 5

        Select Case a
            Case 1
                Console.WriteLine("one")
            Case Else
                Console.WriteLine("other")
        End Select

        Try
            Await Task.Delay(10)
        Catch ex As Exception When a > 0
            Throw
        Finally
            Me.Dispose()
        End Try

        Return If(a Is Nothing, 0, a Xor 1)
    End Function

    Public Sub Dispose() Implements IDisposable.Dispose
    End Sub
End Class
```

VHDL

Monday, October 26, 2026 5:25 PM

```vhdl
-- plain line comment

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Counter is
    generic (
        WIDTH : integer := 8;
        DELAY : time := 10 ns;
        RATIO : real := 3.14;
        BIG   : natural := 1_000_000
    );
    port (
        clk   : in  std_logic;
        rst   : in  std_logic;
        en    : in  boolean;
        count : out std_logic_vector(WIDTH - 1 downto 0);
        done  : out bit
    );
end entity Counter;

architecture Behavioral of Counter is
    type state_t is (IDLE, RUN, STOP);
    signal state   : state_t := IDLE;
    signal value   : unsigned(7 downto 0) := (others => '0');
    signal sval    : signed(7 downto 0);
    signal bits    : bit_vector(3 downto 0) := "1010";
    signal hexv    : std_logic_vector(7 downto 0) := X"FF";
    signal binv    : std_logic_vector(3 downto 0) := B"1010";
    signal octv    : std_logic_vector(5 downto 0) := O"77";
    signal ch      : character := 'x';
    signal msg     : string(1 to 5) := "hello";
    signal flag    : boolean := true;
    signal off     : boolean := false;
    signal pos     : positive := 1;
    signal ustd    : std_ulogic;
    constant BASED : integer := 16#FF#;
    constant EXPO  : real := 6.02E23;
    constant BIN2  : integer := 2#1010_0101#;

    function twice(x : integer) return integer is
    begin
        return x * 2;
    end function;
begin

    process (clk, rst)
        variable tmp : integer range 0 to 255;
    begin
        if rst = '1' then
            value <= (others => '0');
        elsif rising_edge(clk) then
            case state is
                when IDLE =>
                    state <= RUN;
                when RUN =>
                    value <= value + 1;
                when others =>
                    null;
            end case;

            for i in 0 to 7 loop
                tmp := tmp + i;
                next when i = 3;
```

```
                exit when i = 6;
            end loop;

            while tmp > 0 loop
                tmp := tmp - 1;
            end loop;
        end if;
    end process;

    count <= std_logic_vector(value) when en else (others => 'Z');
    done  <= '1' after 5 ns;

    gen : for k in 0 to 3 generate
        assert k >= 0 report "bad index" severity error;
    end generate gen;

end architecture Behavioral;
```

Wolfram

Monday, October 26, 2026 5:35 PM

Language Overview—Wolfram Language Documentation

```
CheckForFreeze[] :=
 Module[{temperatureData, minimumTemperature},
  temperatureData =
   WolframAlpha[
    "temperature Urbana IL", {{"TemperatureChart:WeatherData", 1},
     "ComputableData"}];
  minimumTemperature =
   Min[Cases[
     temperatureData, {Append[Take[DateList[], 3] + {0, 0, 1}, ___],
       t_} :> t, \[Infinity]]];
  If[QuantityMagnitude[
     UnitConvert[minimumTemperature, "DegreesCelsius"]] < 0,
   SendFreezeAlertMail[]]
  ]
SendFreezeAlertMail[] :=
 SendMail[
  "To" -> "who@mailprovider.com",
  "From" -> "who@mailprovider.com",
  "Subject" -> "FREEZE ALERT",
  "Body" -> {
    "Freezing weather expected tomorrow in Urbana, IL ...",
    "\n\n",
    WolframAlpha["temperature Urbana, IL",
     IncludePods -> "TemperatureChart:WeatherData",
     AppearanceElements -> {"Pods"},
     TimeConstraint -> {30, Automatic, Automatic, Automatic}],
    "\n\n",
    " http://www.wolframalpha.com/input/?i=temperature+Urbana+IL"
    },
  "Server" -> "mail.mailprovider.com",
  "UserName" -> "who",
  "Password" -> "xxxxxxx",
  "EncryptionProtocol" -> "SSL"
  ]
obj = CloudSave[CheckForFreeze]
With[{file = obj},
 CloudDeploy[
  ScheduledTask[CloudGet[file]; CheckForFreeze[],
   DateObject[{_, _, _, 18, _, _}]]]]


(* plain comment on one line *)
(*
   multi-line comment
*)

(* numbers *)
a = 42;
b = 3.14;
c = 6.02*^23;
d = 1.5;

(* strings *)
s1 = "plain \"quoted\" string";
s2 = 'single quoted';

(* system variables *)
$Version
$MachineName
$RecursionLimit = 1024;

(* literals and symbols *)
```

```
t = True; f = False; n = Null; i = Infinity; p = Pi; e = E;

(* function calls and brackets *)
f[x_] := x^2 + Sin[x] Cos[x]
g[x_, y_] := Plot[Sin[x y], {x, 0, 2 Pi}, PlotRange -> All]
lst = {1, 2, 3, 4};
part = lst[[2]];
Map[f, lst]
Table[i^2, {i, 1, 10}]
Select[lst, EvenQ]
Integrate[x^2, {x, 0, 1}]

(* control flow *)
If[a > 0 && t, Print["positive"], Print["negative"]]
Do[Print[k], {k, 1, 3}]
While[a > 0, a--]
For[j = 0, j < 5, j++, Print[j]]
Which[a == 1, "one", a == 2, "two", True, "other"]
Switch[a, 1, "one", _, "other"]
Module[{x = 1}, x + 1]
Block[{y = 2}, y^2]
Return[a]; Break[]; Continue[];

(* operators *)
x = y + z - w * v / u;
x += 1; x -= 1; x *= 2; x /= 2; x++; x--;
```

XML

Monday, October 26, 2026 4:54 PM

```xml
<?xml version="1.0" encoding="UTF-8" standalone='yes'?>
<?xml-stylesheet type="text/xsl" href="style.xsl"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<!-- single line comment -->
<!--
  multi-line comment
-->
<ns:root xmlns:ns="http://example.com/ns" xmlns="http://example.com/default" version='1.0'>
  <item id="1" name="first &amp; foremost" enabled>
    Text content with &lt;entities&gt; &#169;
  </item>
  <ns:item ns:attr="value" other-attr='single'/>
  <empty/>
  <data><![CDATA[ raw <not> a tag & stuff ]]></data>
  <multi
      line="attribute"
      second='two'>
    <nested.name>x</nested.name>
  </multi>
</ns:root>
```

YAML

Monday, October 26, 2026 5:00 PM

```yaml
# plain comment
---
name: Sample Project
version: 1.2.3
count: 42
ratio: 3.14
enabled: true
disabled: false
nothing: null
tilde: ~
quoted: "double \"quoted\" string"
single: 'single quoted'
url: https://example.com/path?x=1#frag
anchors:
  base: &base
    key: value
  derived:
    <<: *base
    extra: 1
list:
  - first
  - second
  - 3
  - { inline: map, n: 2 }
  - [a, b, c]
nested:
  deeper:
    deepest: "value"   # trailing comment
multi: |
  literal block
  with lines
folded: >
  folded block
  text
"quoted key": value
? complex key
: complex value
```